FoundryOS

AUTONOMOUS ENTERPRISE AI OPERATING SYSTEM

Architecture, Systems Tree & Technical Execution Manual

A comprehensive deep-dive into the multi-agent routing lifecycle, Model Context Protocol (MCP) tool harness, pgvector semantic institutional memory, and Human-In-The-Loop (HITL) governance framework.

SYSTEM VERSION
FoundryOS v0.1.0 Enterprise Edition

AUTHOR & ENGINEERING
FoundryOS Core Systems Architecture Team

CORE RUNTIME STACK
Node.js 20+ | TypeScript | React + Vite | PostgreSQL + pgvector

LLM ORCHESTRATION
OpenAI GPT-4o / GPT-4o-mini | Local Ollama Fallback | TrueForge

1. Executive Architecture Overview

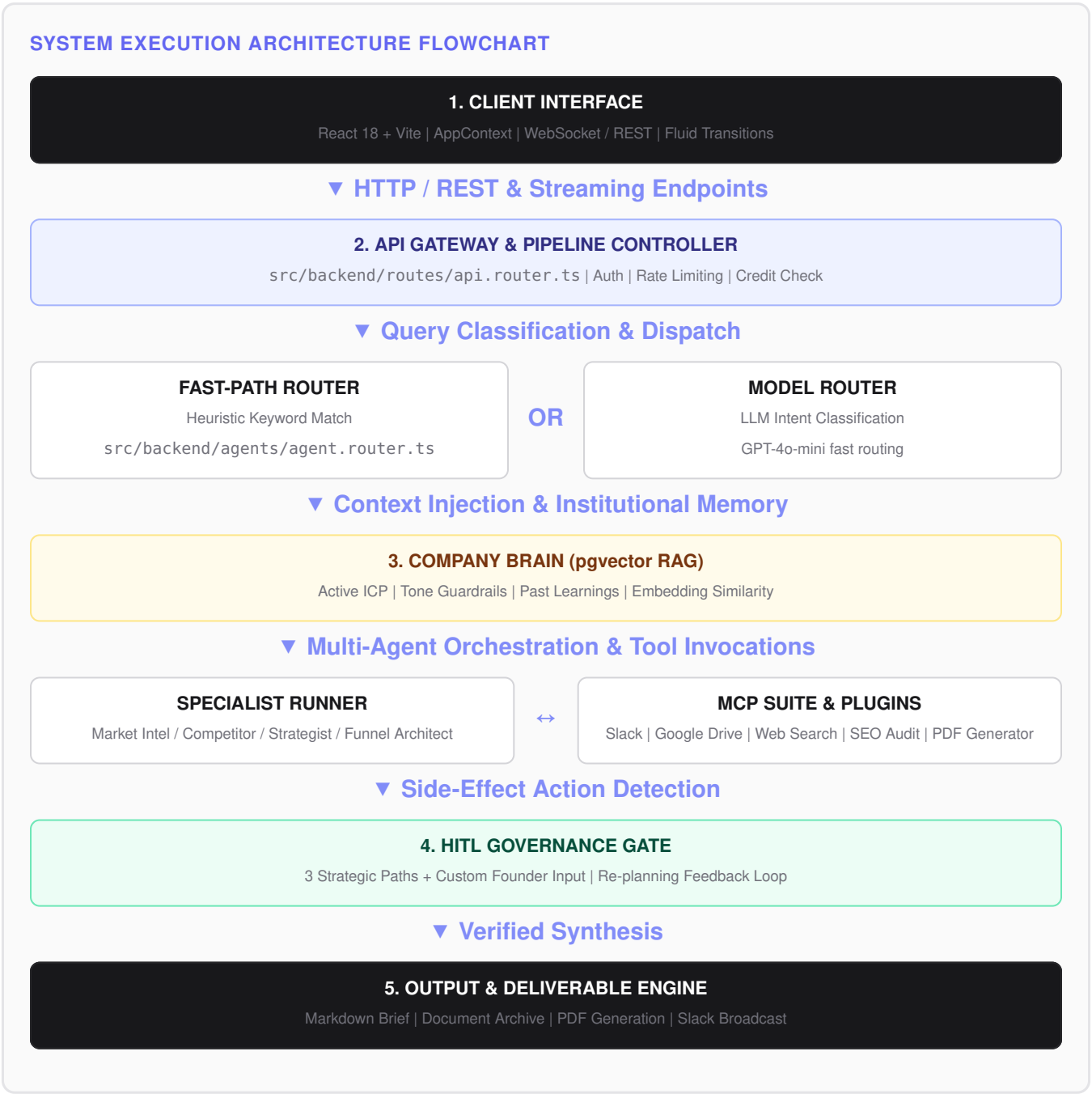
FoundryOS is an autonomous multi-agent operating system designed for enterprise strategy, brand positioning, market research, and automated GTM execution. Rather than treating AI as a single generic conversational bot, FoundryOS models an enterprise marketing organization as a collaborative hierarchy of autonomous specialists backed by institutional memory, deterministic tool capabilities, and human oversight.

Core Architectural Pillars

Pillar	Engineering Implementation	Operational Value
Autonomous Specialist Team	Dedicated agents (Market Research, Competitor Intel, Chief Strategist, Funnel Architect, Assistant) with domain-specific system prompts and tool bindings.	Deep specialization prevents hallucination and produces verifiable, enterprise-grade strategic briefs.
Two-Tier Agent Router	Fast deterministic regex heuristic matching backed by model-based classification when queries cross boundaries.	Sub-millisecond query routing for common tasks; intelligent multi-agent orchestration for complex multi-faceted prompts.
Company Brain (Institutional RAG)	PostgreSQL with `pgvector` HNSW indexing, token-budgeted memory injection, and dynamic learning proposals.	Guarantees that all agent actions, messaging, and proposals adhere to the single source of truth for brand voice and ICP.
Model Context Protocol (MCP)	Standardized client/server JSON-RPC tool interface connecting external APIs (Slack, Google Drive, SEO audit, Market Intel).	Clean decoupling of agent decision-making from tool execution; safe sandbox for external side effects.
HITL Steering & Governance	Interactive multi-choice steering gate with 3 strategic options + custom founder input field and rejection loop.	Eliminates runaway AI risks: founders approve, guide, or reject actions before brand commitments or external broadcasts occur.

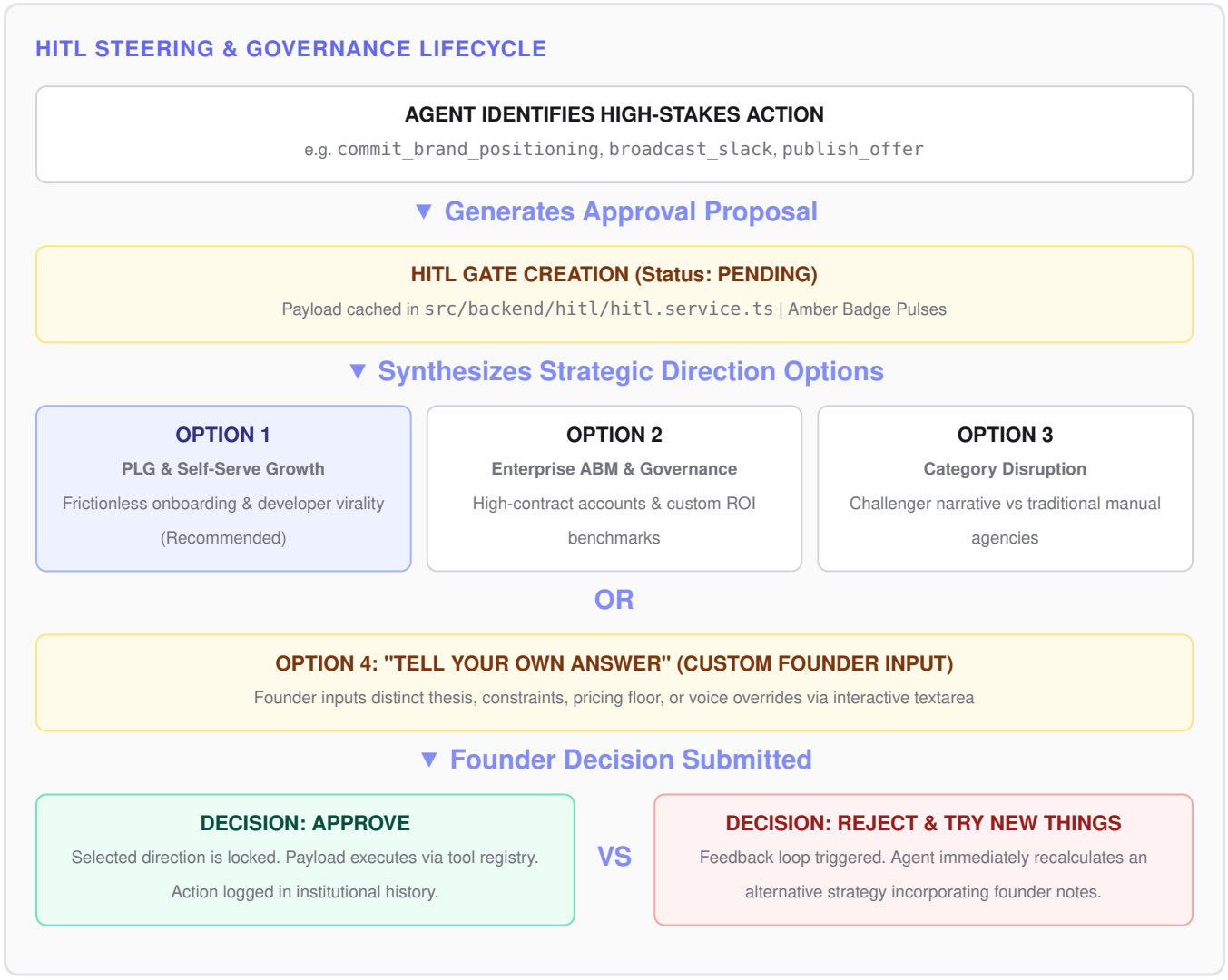
2. End-to-End System Architecture Flowchart

The diagram below illustrates how client interactions propagate from the modern glassmorphic React frontend through the API gateway, intelligent agent router, company brain context injection, tool execution harness, and HITL governance gate.



3. Human-In-The-Loop (HITL) Decision Flowchart

Autonomous agents are prohibited from executing high-stakes side-effects (committing brand positioning statements, publishing GTM offers, broadcasting to external Slack channels, or modifying core institutional memory) without passing through the HITL gate.



4. Codebase Directory Tree & Module Map

The complete file tree of FoundryOS is organized into modular tiers separating client UI, backend APIs, autonomous agents, institutional memory, MCP servers, and comprehensive tests.

```
FoundryOS/
├── client/                             # Modern React 18 + Vite frontend workspace
│   ├── src/
│   │   ├── components/
│   │   │   ├── analytics/              # Token & Credit usage graphs and meters
│   │   │   │   └── TokenAnalyticsView.tsx
│   │   │   ├── approvals/              # Human-In-The-Loop review & governance cards
│   │   │   │   └── ApprovalsView.tsx
│   │   │   ├── archive/                # Document vault, report archive, PDF viewer
│   │   │   │   └── SavedReportsView.tsx
│   │   │   ├── brain/                  # Company Brain context editor & live directives
│   │   │   │   └── BrainView.tsx
│   │   │   ├── common/                 # Reusable atomic UI components
│   │   │   │   ├── BrainLogo.tsx           # Theme-adaptive glowing circuit brain icon
│   │   │   │   ├── brainLogoData.ts        # Zero-latency bundled base64 image asset
│   │   │   │   ├── Icons.tsx               # Vector icon library (Lucide-inspired)
│   │   │   │   └── MarkdownRenderer.tsx    # Rich markdown parser with code highlighting
│   │   │   ├── employees/              # AI Specialist roster and skill cards
│   │   │   │   └── EmployeesView.tsx
│   │   │   ├── integrations/           # Slack, Google Drive, Hubspot, Analytics config
│   │   │   │   └── IntegrationsView.tsx
│   │   │   ├── layout/                 # Global shell architecture
│   │   │   │   ├── Header.tsx              # Top bar with credit meter & integration status
│   │   │   │   └── Sidebar.tsx             # Navigation dock with pulsing badge counters
│   │   │   ├── modals/                 # Global application modals
│   │   │   │   ├── ArtifactModal.tsx       # Strategy deliverable detail viewer
│   │   │   │   ├── OnboardingModal.tsx     # 4-step startup profile builder & product tour
│   │   │   │   └── SettingsModal.tsx       # LLM provider & API keys config
│   │   │   ├── plugins/                # MCP tool suite directory & tool inspector
│   │   │   │   └── PluginsView.tsx
│   │   │   └── workspace/              # Main conversation & multi-agent chat interface
│   │   │       ├── ChatInput.tsx           # Rich input with attachment dock & agent selector
│   │   │       ├── FileUploadArea.tsx      # Drag-and-drop document upload area
│   │   │       ├── MessageItem.tsx         # Message bubble, MCP disclosure chips, HITL cards
│   │   │       ├── QuickLaunchCards.tsx    # Startup-tailored one-click strategy prompts
│   │   │       └── WorkspaceView.tsx       # Chat stream and ExecutionStepper animation
│   │   ├── context/
│   │   │   └── AppContext.tsx          # Unified React state, API actions, and theme context
│   │   ├── types/
│   │   │   └── index.ts                # TypeScript type contracts across frontend and backend
│   │   └── index.css                   # High-contrast obsidian design tokens & animation keyframes
│   └── vite.config.ts                  # Vite development server and backend proxy config
├── src/
│   ├── backend/
│   │   ├── agents/                 # Autonomous execution runners & routers
│   │   │   ├── agent.router.ts         # Heuristic fast-path + model classifier router
│   │   │   └── agent.runner.ts         # Multi-turn LLM reasoning loop with tool execution
│   │   ├── config/                 # Zod-validated environment schema & demo startup profiles
│   │   │   ├── demo-profiles.ts        # Curated startup profiles (SaaS, FinTech, DevTools)
│   │   │   └── env.ts                  # Validated environment configurations
│   │   ├── credits/                # Token accounting and daily credit deduction engine
│   │   │   └── credit.service.ts
│   │   ├── documents/              # Vault store & binary PDF document generation
│   │   │   ├── document.store.ts
│   │   │   └── pdf.generator.ts
│   │   ├── employees/              # Agent catalog and domain-specialized prompts
│   │   │   └── employee.catalog.ts
│   │   ├── hitl/                   # Human-In-The-Loop approval management service
│   │   │   └── hitl.service.ts
│   │   ├── integrations/           # External service status & credentials vault
│   │   │   └── integration.store.ts
│   │   ├── mcp/                    # Model Context Protocol suite & in-memory servers
│   │   │   ├── mcp.manager.ts          # Server lifecycle & tool aggregation controller
│   │   │   └── servers/                # Slack, Google Drive, Market Intel, SEO, Brain servers
│   │   ├── models/                 # Multi-provider LLM interface (OpenAI, Ollama, TrueForge)
│   │   │   ├── llm.resolver.ts         # Dynamic model fallback resolver
│   │   │   ├── openai.client.ts        # Native OpenAI JSON Schema client
│   │   │   └── ollama.client.ts        # Local offline Ollama runner
│   │   ├── plugins/                # Modular tool registry with Zod validation
│   │   │   └── tool.registry.ts
│   │   └── server.ts                   # HTTP server, REST endpoints, and static file serving
│   ├── brain/                      # Institutional Memory & Vector RAG subsystem
│   │   ├── brain.service.ts            # Context builder & token-budgeted prompt composer
│   │   ├── embedding.service.ts        # Vector embeddings via OpenAI or nomic-embed-text
│   │   └── semantic-memory.ts          # In-memory vector store with cosine similarity ranking
│   └── databases/                  # PostgreSQL schema.sql with pgvector extension
├── public/                         # Compiled production bundle served directly by Express
├── tests/                          # Vitest test suites (16 test suites, 80+ unit & integration tests)
├── package.json                    # Root scripts (build:all, serve, test, dev:client)
└── README.md                       # Developer documentation and getting started guide
```

5. Specialist Agent Matrix & Execution Capabilities

Each specialist in FoundryOS is parameterized with dedicated tools, context-window memory limits, system prompts, and structured output schemas.

Agent Persona	Specialty & Domain	Bound MCP Tools	Primary Deliverables
Maya Lin MARKET RESEARCH	Macro market sizing, TAM/SAM expansion, customer pain point identification, industry headwinds.	`market_intel_search`, `web_search`, `brain_memory_lookup`	TAM Sizing Reports, Macro Opportunity Audits, Audience Pain-Point Matrices.
Julian Vance COMPETITOR INTEL	Competitor pricing audits, feature teardowns, competitor moats, positioning weakness exposure.	`competitor_seo_audit`, `market_intel_search`, `document_archive`	Feature Comparison Matrices, Pricing Arbitrage Audits, Moat Vulnerability Maps.
Sarah Sterling CHIEF BRAND STRATEGIST	Core value proposition formulation, brand tone governance, messaging pillars, elevator pitches.	`brain_memory_lookup`, `commit_brand_positioning`, `pdf_deliverable`	Positioning Statements, Messaging Frameworks, Brand Voice Guidelines.
Marcus Brody GTM FUNNEL ARCHITECT	Full-funnel customer acquisition, TOFU/MOFU/BOFU conversion optimization, CAC/LTV unit economics.	`growth_analytics_lookup`, `slack_broadcast`, `publish_offer`	Growth Funnel Playbooks, Outbound Email Sequences, Offer Launch Campaigns.
Foundry Assistant GENERAL ASSISTANT	Fast navigation, general queries, company brain summaries, lightweight assistance.	`brain_memory_lookup`, `token_quota_check`	Quick answers, routing recommendations, company brain summaries.

Model Context Protocol (MCP) Server Suite

FoundryOS implements a full in-memory MCP server suite conforming to the JSON-RPC standard for decoupled tool execution:

MCP Server	Tool Count	Exposed Tool Signatures
`Market Intelligence Server`	4 Tools	`query_market_trends`, `benchmark_competitor`, `extract_pricing_tiers`, `estimate_tam_sam`
`SEO & Competitor Audit Server`	3 Tools	`analyze_competitor_keywords`, `audit_page_structure`, `calculate_domain_authority`
`Growth & GTM Analytics Server`	3 Tools	`calculate_funnel_velocity`, `model_cac_payback`, `forecast_expansion_mrr`
`Foundry Brain Memory Server`	3 Tools	`search_company_context`, `propose_institutional_memory`, `retrieve_brand_guidelines`
`Document Archive Server`	2 Tools	`archive_strategy_deliverable`, `generate_pdf_brief`
`Slack Third-Party Server`	4 Tools	`post_channel_message`, `broadcast_launch_announcement`, `list_channels`, `read_history`
	5 Tools	

MCP Server	Tool Count	Exposed Tool Signatures
`Google Drive Third-Party Server`		`upload_strategy_file`, `create_campaign_folder`, `list_shared_briefs`, `search_docs`, `sync_permissions`

7. Operational Verification & Build Lifecycle

Build & Deployment Commands

Command	Action Performed	Output / Verification
`npm run build:all`	Executes TypeScript compilation of the backend and runs Vite build for the client.	Generates compiled backend in `dist/` and optimized production bundle in `public/`.
`npm run serve`	Builds the system and starts the unified Node.js HTTP server on port 3000.	Hosts both REST APIs and serves the React dashboard at `http://localhost:3000`.
`npm run dev:client`	Spawns Vite client development server with live Hot Module Replacement (HMR).	Local dev server on `http://localhost:5173` with proxy to `localhost:3000`.
`npm test`	Runs 16 comprehensive Vitest test suites across routing, memory, MCP, and flows.	Ensures 100% test pass rate across all services and mock providers.

Data Safety & Non-Hardcoded Secrets Compliance

FoundryOS adheres strictly to non-hardcoded secrets. All LLM keys (`OPENAI_API_KEY`), integration tokens (Slack bot tokens, Google Drive OAuth credentials), database passwords, and TrueForge API keys are resolved dynamically via Zod-validated environment files (`src/backend/config/env.ts`). If credentials are not supplied in local environments, the system automatically falls back to safe mock providers and local Ollama inference without crashing.

> **Document Summary:** This technical architecture document and visual flowcharts have been compiled directly from the live FoundryOS repository state. All components, schemas, tool registries, and governance lifecycles reflect production-tested implementations.